

Technical Assessment: Real-Time High-Traffic Inventory System

Project Overview

You are building the backend and frontend for a "Limited Edition Sneaker Drop." Because items are in high demand, users need to see stock updates in real-time, and they must be able to "Reserve" an item for a limited window to complete their checkout.

Stack Requirements:

- **Frontend:** React (State management of your choice).
- **Backend:** Node.js + Express.
- **Database:** PostgreSQL
- **Real-time:** WebSockets ([Socket.io](https://socket.io/)).
- **ORM:** Prisma

Core Features to Implement

1. TI

2. The "Atomic Reservation" System

- Users can click "Reserve" on an item. This should temporarily decrease the available stock by 1.
- **Constraint:** You must prevent "**Overselling.**" If 100 users click "Reserve" at the exact same millisecond for the last 1 item, **only 1 user** should succeed.
- The reservation lasts for **60 seconds**.

3. The "Stock Recovery" Mechanism

- If a user does not click "Complete Purchase" within the 60-second window, the reservation expires.
- The system must automatically return that 1 unit back to the "Available Stock" and notify all connected clients via WebSockets.

4. The "Purchase" Flow

- Users can only purchase an item they have currently reserved.
- Upon purchase, the stock is permanently deducted.

5. The "Merch Drop" API (No Admin UI Required)

We do not need a visual dashboard for admins. However, we need a robust **API endpoint** to initialize a new merch drop.

- **The Task:** Create an API to create a new "Drop" (e.g., "Air Jordan 1 - 100 units").
- **What we are looking for:** How do you structure the data? Do you handle timestamps (when the drop starts)? How do you initialize the stock?
- **Drop** Table is implied

6. The Feature Addition: "Drop Activity Feed."

New Requirement: On the main dashboard, each Merch Drop must display the **top 3 most recent successful purchasers** (displaying their usernames) directly on the product card.

- **The Data:** You will need a **Users** table and a **Purchases** table.
- **The Goal:** When the frontend requests the list of active Merch Drops, the API response must include the drop details *plus* the nested list of the 3 latest purchasers for *each* drop.

UI & Design Requirements

- **Function over Form:** We are evaluating your **engineering skills**, not your graphic design skills. The UI does not need to be "beautiful," but it must be **clean, usable, and functional**.
- **The "Don't Make it Messy" Rule:** Avoid unstyled HTML or broken layouts. A simple, professional layout using a component library (like Tailwind CSS, Bootstrap, or Shadcn/UI) is perfectly sufficient.
- **Focus Areas:** We care more about **UI Feedback** than UI aesthetics. For example:
 - Does the "Like" or "Reserve" button show a loading state?
 - Does a toast notification appear if the database throws a concurrency error?
 - Is the "Stock Count" clearly visible when it updates in real-time?
- **Constraint:** Do not spend more than 1 hour on styling. Your time is better spent on the Postgres schema, race-condition handling, and WebSocket logic.

Submission Deliverables

1. **GitHub Repository Link.**

2. **A README.md file** explaining:
 - How to run the app (including SQL schema setup).
 - **Architecture Choice**: How did you handle the 60-second expiration logic?
 - **Concurrency**: How did you prevent multiple users from claiming the same last item?
3. **A 2-minute Loom/Video demo** showing two browser windows side-by-side demonstrating the real-time stock sync.

Important Note: Deployment & Bonus Points

We highly value developers who can take an application from local development to a live production environment.

- **Bonus Points**: You will receive extra credit if you host your application online and provide a live URL in your submission.
- **Recommended Stack for Hosting**:
 - **Frontend & Backend**: [Vercel](#) (Vercel supports both React and Node.js/Express. For the backend, ensure your entry point is in an `/api` folder or configured via a `vercel.json` file).
 - **Postgres Database**: [Neon xDB](#) (A serverless PostgreSQL provider that offers a generous free tier and integrates perfectly with Vercel).
- **Environment Variables**: Please ensure you do **not** commit your database credentials or `.env` files to GitHub. **Instead, use the environment variable settings in Vercel to securely connect your app to Neon**